

Disciplina: DCA 3703 - PROGRAMAÇÃO PARALELA

Professor: Samuel Xavier De Souza

Nome: José Felix Rodrigues Anselmo

Tarefa 6

1 - Enunciado

Implemente em C a estimativa estocástica de π . Paralelize com `#pragma omp parallel for` e explique o resultado incorreto. Corrija a condição de corrida utilizando o `#pragma omp critical` e reestruturando com `#pragma omp parallel` seguido de `#pragma omp for` e aplicando as cláusulas `private`, `firstprivate`, `lastprivate` e `shared`. Teste diferentes combinações e explique como cada cláusula afeta o comportamento do programa. Comente também como a cláusula `default(none)` pode ajudar a tornar o escopo mais claro em programas complexos.

2 - Metodologia

Esse experimento é bem interessante pois ele requer várias análises a partir de uma implementação, inicialmente para começo de conversa foi gerado um código que funciona de maneira sequencial que calcula π de maneira estocástica através do método de monte carlos, usando a mesma implementação foi feita a implementação do código em paralelo seguindo o que era pedido na tarefa, os códigos foram compilados da seguinte forma.

```
gcc sequencepi.c -o sequencepi
time ./sequencepi
gcc -fopenmp parallempi.c -o parallempi
export OMP_NUM_THREADS=12; time ./parallempi
```

2.3 - Ambiente de Execução

Componente	Especificação
Notebook	Lenovo IdeaPad 3 15ALC6
SO	Ubuntu 26.04 LTS
Kernel	Linux 7.0.0-29-generic
CPU	AMD Ryzen 5 5500U — 6C/12T
Arquitetura	x86_64
RAM	9,05 GiB
Desktop	GNOME 50.1
Shell	Zsh 5.9
Terminal	Kitty 0.45.0
Compilador	GCC

3 - Código Fonte

Para auxilia durante a verificação nos códigos de teste das cláusulas foi anexada apenas os trechos onde houveram mudanças no código fonte:

3.1 - sequencepi.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

static double medirTempo(void) {
    struct timespec ts;
```

```
clock_gettime(CLOCK_MONOTONIC, &ts);  
return ts.tv_sec + ts.tv_nsec * 1e-9;  
}  
  
int main(){  
    long long N = 10000000;  
    long long pontos_dentro = 0;  
    double x,y;  
  
    srand((unsigned int)time(NULL));  
    double inicio = medirTempo();  
    for(long long i = 0; i < N; i++){  
        x = (double)rand()/RAND_MAX;  
        y = (double)rand()/RAND_MAX;  
  
        if((x*x)+(y*y) <= 1.0){  
            pontos_dentro++;  
        }  
    }  
    double fim = medirTempo();  
  
    double pi_estimado = 4.0 * ((double)pontos_dentro / N);  
  
    printf("Tempo Medido: %f (s) \n", fim - inicio);  
    printf("Pontos sorteados: %lld\n", N);  
    printf("Pontos dentro do círculo: %lld\n", pontos_dentro);  
    printf("Valor estimado de Pi: %f\n", pi_estimado);  
  
    return 0;  
}
```

3.2 - parallelpi_s.c(sem critical)

```
#pragma omp parallel for
for(long long i = 0; i < N; i++){
    ...
}
```

3.3 - parallelpi.c(omp critical)

```
if((x*x)+(y*y) <= 1.0){
    #pragma omp critical
    pontos_dentro++;
}
```

3.4 - parallelpi_shared(utilizando a clausula shared)

```
#pragma omp parallel shared(N, pontos_dentro)
....
}
```

3.5- parallel.c(utilizando private)

```
#pragma omp parallel private(x,y)
....
}
```

3.6 parallel.c(lastprivate)

```
long long ultimo = -1;
...
#pragma omp parallel
{
    long long local_dento_c = 0;

    #pragma omp for lastprivate(ultimo)
    for(long long i = 0; i < N; i++){
        //Gear as coordenadas de x e y entre 0 e 1
        x = (double)rand()/RAND_MAX;
        y = (double)rand()/RAND_MAX;

        if ((x * x) + (y * y) <= 1.0)
        {
            local_dento_c++;
        }
        ultimo = i;
    }
    #pragma omp critical
    pontos_dentro += local_dento_c;
}
```

3.7- parallel.c(todas as cláusulas do enunciado aplicadas)

```
#include <stdio.h>

#include <stdlib.h>

#include <omp.h>

#include <time.h>

int main(){

    long long N = 10000000;

    long long pontos_dentro = 0;

    long long ultimo = -1;

    double x,y;

    srand((unsigned int)time(NULL));

    double inicio = omp_get_wtime();

    #pragma omp parallel default(none) shared(N, pontos_dentro, ultimo) firstprivate(x,y)
    {

        long long local_dento_c = 0;

        #pragma omp for lastprivate(ultimo)
        for(long long i = 0; i < N; i++){

            //Gear as coordenadas de x e y entre 0 e 1

            x = (double)rand()/RAND_MAX;

            y = (double)rand()/RAND_MAX;

            if ((x * x) + (y * y) <= 1.0)
            {

                local_dento_c++;

            }

            ultimo = i;

        }

        #pragma omp critical
```

```

    pontos_dentro += local_dento_c;
}

double fim = omp_get_wtime();

double pi_estimado = 4.0 * ((double)pontos_dentro / N);

printf("Pontos sorteados: %lld\n", N);
printf("Pontos dentro do círculo: %lld\n", pontos_dentro);
printf("Valor estimado de Pi: %f\n", pi_estimado);
printf("Tempo Medido: %f (s) \n", fim - inicio);

return 0;
}

```

4 - Resultado

O experimento foi iniciado com a criação do código sequencial que calcula o valor de pi de maneira estocástica e que serve como base para a implementação posterior destacada na tarefa.



```

> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_6
> ./sequencepi
Tempo Medido: 0.157701 (s)
Pontos sorteados: 10000000
Pontos dentro do círculo: 7852856
Valor estimado de Pi: 3.141142

```

Figura 1: Execução de forma sequencial

Após os testes com o código sequencial foi feita a reestruturação do código de forma paralela usando o `#pragma parallel for`:

```
> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_6 _ □ X
> export OMP_NUM_THREADS=12; time ./parallempi_s
Pontos sorteados: 100000000
Pontos dentro do círculo: 7810211
Valor estimado de Pi: 3.124084
Tempo Medido: 2.168683 (s)
./parallempi_s 2,85s user 16,33s system 882% cpu 2,173 total
```

Figura 2: Saída código paralelo sem critical

Alterando no código, dividindo o `#pragma omp parallel` e `#pragma omp parallel`, e usando o `#pragma omp critical` resolvendo problema da zona critica.

```
> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_6 _ □ X
> export OMP_NUM_THREADS=12; time ./parallempi_c
Pontos sorteados: 100000000
Pontos dentro do círculo: 7857063
Valor estimado de Pi: 3.142825
Tempo Medido: 2.566221 (s)
./parallempi_c 4,03s user 18,72s system 884% cpu 2,570 total
```

Figura 3: parallel.c com critical

E por fim a última implementação usando todas as cláusulas referidas no enunciado:

```
> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_6 _ □ X
> export OMP_NUM_THREADS=12; time ./parallempi
Pontos sorteados: 100000000
Pontos dentro do círculo: 7852936
Valor estimado de Pi: 3.141174
Tempo Medido: 2.159433 (s)
./parallempi 2,68s user 16,33s system 878% cpu 2,164 total
```

Figura 4: parallel.c versão final

Abaixo foi reunido os resultados dos testes feitos fazendo a mudanças clausulas apenas das clausulas nos código de maneira que não interfira no desempenho, para que seja possível a observação do

Tabela 1: Resultados dos experimentos

Compilação	Tempo Medido (s)	Pontos dentro do Circulo	Valor estimado de pi
Sequencial	0,157701	7852856	3,141142
Paralelo	2,168683	7810211	3,124084
Paralelo com Critical	2,5666221	7857063	3,142825
Paralelo com private	2,626882	7855701	3,142280
Paralelo com shared	2,552596	7861189	3,144476
Paralelo com lastprivate	2,244016	7847444	3,138978
Paralelo Final	2,153821	7852853	3,141141

5 - Conclusão

Os testes realizados evidenciam, em primeiro lugar, que a simples paralelização de um algoritmo não garante que o resultado seja obtido corretamente e nem que irá ter uma melhora no desempenho. A versão sequencial levou 0,157701s e produziu uma estimativa de $\pi = 3,141142$, próxima do valor real $\pi \approx 3,141592653$. Já a versão paralela sem qualquer proteção de acesso concorrente apresentou tempo muito superior (2,168683s) e uma estimativa mais distante ($\pi = 3,124084$), evidenciando o efeito da condição de corrida sobre a variável pontos_dentro múltiplas threads lendo e escrevendo o mesmo endereço de memória simultaneamente perdendo incrementos, gerando uma contagem de pontos inconsistente e, conseqüentemente, uma estimativa de π incorreta.

A introdução do `#pragma omp critical` resolveu esse problema de corretude a estimativa da versão "Paralelo com Critical" ($\pi = 3,142825$ valor mais próximo ao obtido no código sequencial em outras execuções), isso ocorre pois o incremento de pontos_dentro passou a

ser uma operação atômica entre as threads porém o tempo de execução não melhorou (2,566221 s), na verdade ficou muito próximo da versão com corrida. Isso mostra que a correção da condição de corrida resolve o problema de corretude, mas não necessariamente o de desempenho a sincronização imposta pelo critical a cada iteração do laço introduz um custo de contenção comparável ao ganho teórico obtido pela distribuição do trabalho entre as 12 threads.

Nas variantes que testam as cláusulas de escopo (private, shared, lastprivate), os tempos observados ficaram na mesma ordem de grandeza dos anteriores (2,159433s a 2,626882s), com pequenas variações que refletem que podem ser atribuídas ao sistema operacional entre execuções, e não um efeito estrutural de desempenho de cada cláusula isoladamente. Isso é esperado: private, shared, firstprivate e lastprivate controlam onde e como os dados são armazenados e compartilhados entre as threads, o papel dessas cláusulas é, portanto, de corretude e controle de escopo, não de otimização de tempo.

Por fim, o uso do default(none) na versão final mostrou seu valor mais como ferramenta do que de desempenho, ao forçar a classificação explícita de cada variável usada dentro da região paralela, o compilador impede que o programador esqueça de definir o escopo de alguma variável, evitando erros silenciosos de compartilhamento indevido como o que causou a condição de corrida original.